

Unit III: Central Processing Unit

stack organization, instruction formats, addressing modes, data transfer and manipulation, program control, RISC, parallel processing and pipelining.

Stack

Formats

Addressing

Data instructions

Program control

RISC

Parallel

Pipelining

1. Stack organization

A stack is a last-in, first-out (LIFO) list. The item stored last is retrieved first. A register called the **stack pointer (SP)** always holds the address of the top item. Stacks make subroutine calls, interrupts and expression evaluation efficient.

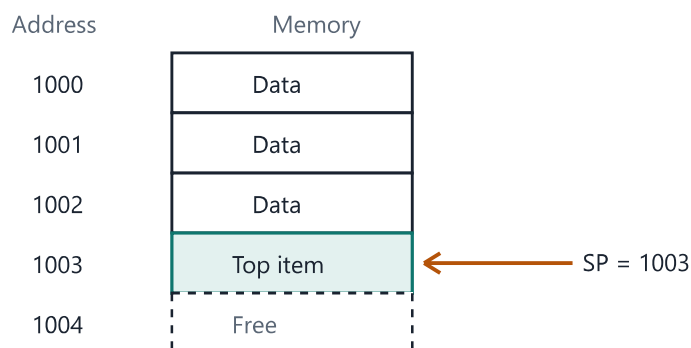
Register stack

A small set of registers (say 64 words) with a 6-bit SP and two flip-flops, FULL and EMPTY.

PUSH: $SP \leftarrow SP+1$; $stack[SP] \leftarrow DR$; if $SP = 0$ then $FULL \leftarrow 1$; $EMPTY \leftarrow 0$. **POP:** $DR \leftarrow stack[SP]$; $SP \leftarrow SP-1$; if $SP = 0$ then $EMPTY \leftarrow 1$; $FULL \leftarrow 0$.

Memory stack

A part of main memory is used as the stack, and SP points into it. By convention it grows toward lower addresses. **PUSH:** $SP \leftarrow SP-1$; $M[SP] \leftarrow DR$. **POP:** $DR \leftarrow M[SP]$; $SP \leftarrow SP+1$. Overflow and underflow are checked against upper and lower limit registers.



PUSH: SP decreases, then write. POP: read, then SP increases.

Memory stack. SP points at the top item, and the stack grows toward lower addresses.

Reverse Polish Notation (postfix)

Infix places the operator between operands ($A+B$), prefix before ($+AB$), postfix after ($AB+$).

Postfix needs no parentheses. Example: $(A+B)*(C+D)$ becomes $AB+CD+*$. To evaluate, scan left to right. Push each operand. On an operator, pop two operands, apply it, and push the result.

Token	Action	Stack (top on right)
3	push	3
4	push	3, 4
+	pop 4,3; push 7	7
2	push	7, 2
5	push	7, 2, 5
+	pop 5,2; push 7	7, 7
*	pop 7,7; push 49	49

The table evaluates $34+25+*$, which is $(3+4)\times(2+5) = 49$.

2. Instruction formats

An instruction is a binary code with three kinds of field: the **operation code** (what to do), the **address field(s)** (where the operands are) and the **mode field** (how to interpret the address).



One instruction word, typically 16 or 32 bits

General instruction format.

By number of address fields

Evaluate $X = (A+B)*(C+D)$. R1 and R2 are registers, T is a temporary memory location.

Type	Program	Notes
Three-address	ADD R1,A,B ADD R2,C,D MUL X,R1,R2	3 instructions, but long ones. Result written to a separate destination.
Two-address	MOV R1,A ADD R1,B MOV R2,C ADD R2,D MUL R1,R2 MOV X,R1	Destination is also a source. Most common in commercial machines.

One-address	LOAD A ADD B STORE T LOAD C ADD D MUL T STORE X	Implied accumulator (AC) is one operand and the destination.
Zero-address	PUSH A PUSH B ADD PUSH C PUSH D ADD MUL POP X	Stack machine. Only PUSH and POP carry an address. Operands are implicit.

Fewer address fields give shorter instructions but longer programs. Fixed-length formats are easier to decode, while variable-length formats save memory.

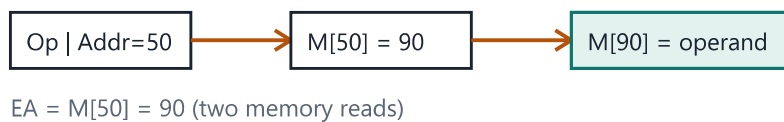
3. Addressing modes

An addressing mode specifies how the operand is found. Modes give programming flexibility (pointers, arrays, loops) and reduce the number of bits in the address field. The **effective address (EA)** is the memory address of the operand.

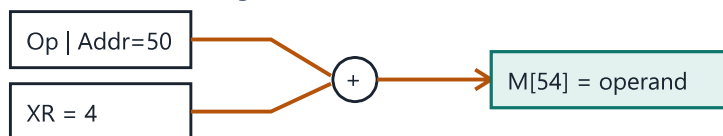
Direct



Indirect



Indexed (also base-register and relative)



EA = 50 + 4 = 54. For relative mode, XR is replaced by PC; for base mode, by the base r

How the effective address is formed in three common modes.

Mode	Effective address / operand	Typical use
Implied	Operand implicit (AC or stack top)	CMA, zero-address stack ops
Immediate	Operand is in the instruction	Constants, initial values
Register	Operand in register R	Fast, short address field
Register indirect	EA = contents of R	Pointers
Autoincrement / autodecrement	EA = R, then $R \leftarrow R \pm 1$	Stepping through arrays
Direct	EA = address field	Simple variables

Indirect	$EA = M[\text{address field}]$	Pointers in memory, jump tables
Relative	$EA = PC + \text{address field}$	Branches, position-independent code
Indexed	$EA = XR + \text{address field}$	Array access, loops
Base register	$EA = BR + \text{address field}$	Relocation, multiprogramming

Worked example: $PC = 200$, instruction at 200 has address field 20 and $XR = 100$. Relative $EA = 201 + 20 = 221$ (PC is already incremented). Indexed $EA = 100 + 20 = 120$.

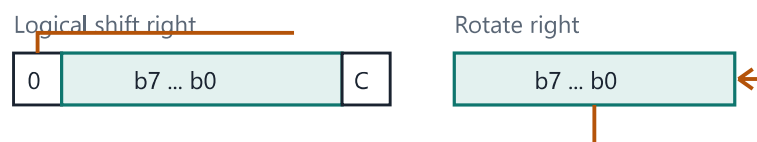
4. Data transfer and manipulation

Data transfer instructions

These copy data between registers, memory and I/O without changing it: **Load** (memory to register), **Store** (register to memory), **Move** (register to register), **Exchange** (swap), **Input/Output** (I/O port), **Push/Pop** (stack).

Data manipulation instructions

Group	Instructions
Arithmetic	Increment, Decrement, Add, Subtract, Multiply, Divide, Add with carry, Subtract with borrow, Negate (2's complement)
Logical and bit	Clear, Complement, AND, OR, XOR, Clear carry, Set carry, Complement carry, Enable/Disable interrupt. AND masks (clears) bits, OR sets bits, XOR complements bits.
Shift	Logical shifts insert 0. Arithmetic shifts keep the sign bit (right shift divides by 2, left shift multiplies by 2). Rotate wraps bits around. Rotate through carry includes the carry flip-flop.



Arithmetic shift right keeps b7 (sign) unchanged and copies it into b6.

Bit movement in shift and rotate instructions.

5. Program control

Program control instructions change the address in the program counter (PC), breaking the default sequential flow.

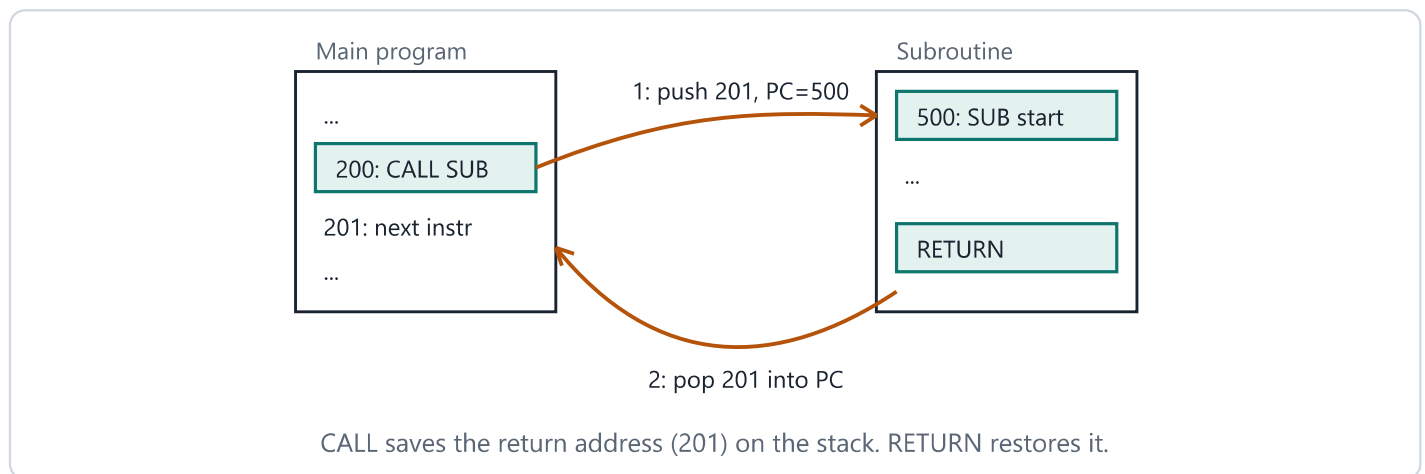
Status bit register (flags)

C carry out of the MSB, **S** sign (MSB of result), **Z** zero result, **V** overflow. Compare (subtract without saving) and Test (AND without saving) set these flags for the next conditional branch.

Mnemonic	Branch if	Condition
BZ / BNZ	zero / not zero	$Z = 1 / Z = 0$
BC / BNC	carry / no carry	$C = 1 / C = 0$
BP / BM	plus / minus	$S = 0 / S = 1$
BV / BNV	overflow / no overflow	$V = 1 / V = 0$
BHI, BLO (unsigned)	higher / lower	based on C and Z
BGT, BLT (signed)	greater / less	based on S, V and Z

Subroutine call and return

CALL: $SP \leftarrow SP - 1$; $M[SP] \leftarrow PC$; $PC \leftarrow \text{subroutine address}$. **RETURN:** $PC \leftarrow M[SP]$; $SP \leftarrow SP + 1$. Using a stack allows nested and recursive calls.



Program interrupts

An interrupt transfers control to a service routine, then resumes the program. The CPU saves the PC and status word (PSW), then loads the routine's address. Types: **external** (I/O device, timer, power failure), **internal / traps** (overflow, divide by zero, illegal opcode, protection violation) and **software** (supervisor call, used by the OS). Interrupts are checked at the end of each instruction cycle.

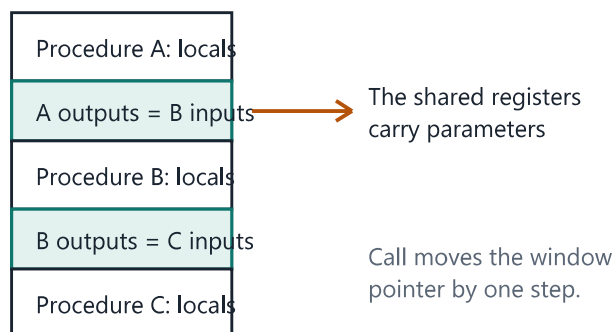
6. Reduced instruction set computer (RISC)

A **CISC** processor has many powerful, variable-length instructions and many addressing modes (VAX, x86). It uses microprogrammed control. A **RISC** processor keeps the instruction set small and simple so it runs fast and pipelines well.

Feature	CISC	RISC
Instruction count	Large (100+)	Small
Length	Variable	Fixed, easy to decode
Addressing modes	Many	Few
Memory access	Many instructions	Only LOAD and STORE
Cycles	Several per instruction	Mostly one
Control unit	Microprogrammed	Hardwired
Registers	Few	Many (large register file)
Emphasis	Hardware complexity	Compiler and pipelining

Overlapped register windows

Procedure calls normally cost many memory accesses to save and restore registers. RISC avoids this with a large register file divided into **windows**. Each procedure gets a window of local registers. Adjacent windows overlap, so the caller's *output* registers are the callee's *input* registers and parameters are passed with no copying.



Overlap gives the caller and callee the same physical registers.

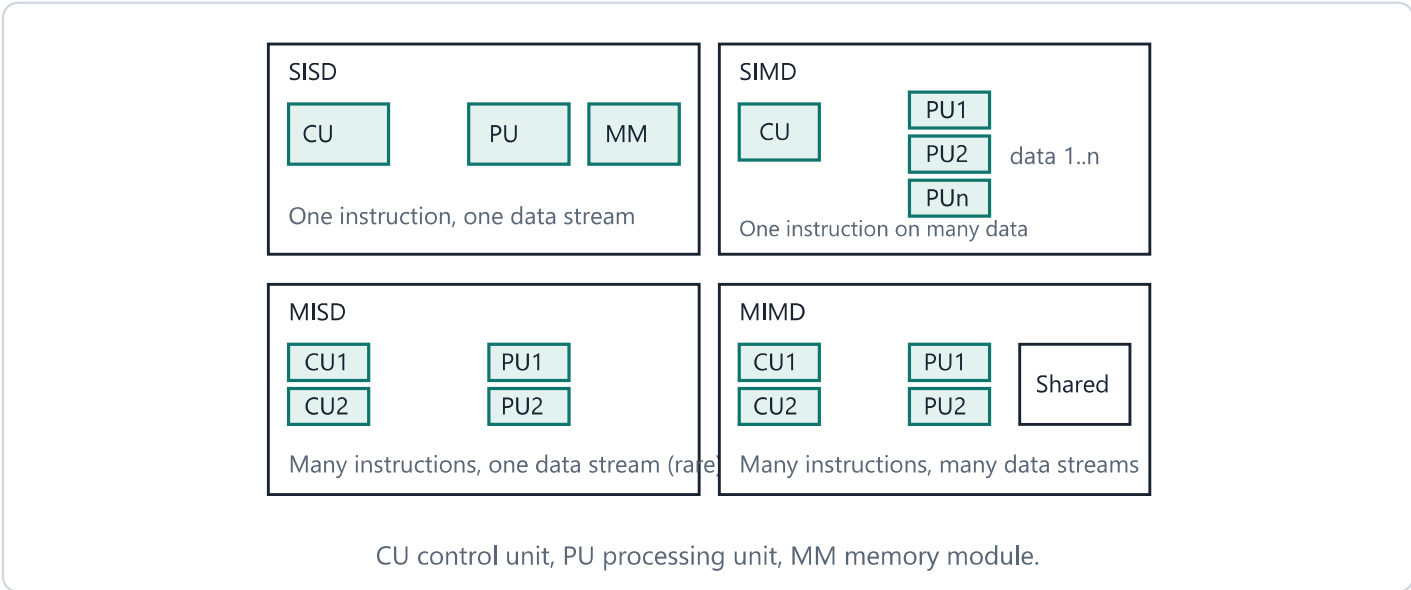
Other RISC ideas

Because RISC pipelines suffer from data and branch conflicts, the compiler helps. **Delayed load:** the compiler places an unrelated instruction (or NOP) after a LOAD so the pipeline does not wait for the data. **Delayed branch:** the instruction after a branch always executes, so the compiler moves a useful instruction into that slot instead of a NOP.

7. Parallel processing

Parallel processing means performing data-processing tasks concurrently to raise speed. It can occur inside one CPU (several functional units such as adder, multiplier, logic unit and shifter working at once, or pipelining) or across several processors.

Flynn's classification



Class	Meaning	Examples
SISD	Single instruction, single data	Classic uniprocessor
SIMD	Single instruction, multiple data	Array and vector processors, GPUs
MISD	Multiple instruction, single data	Rarely built (fault-tolerant designs)
MIMD	Multiple instruction, multiple data	Multiprocessors (shared memory), multicomputers (message passing)

8. Pipelining

Pipelining splits a task into sub-operations, each executed in a dedicated **segment**. Registers between segments hold intermediate results, so different tasks occupy different segments at the

same moment, like an assembly line.

Space-time diagram

A 4-segment pipeline running 6 tasks (T1 to T6):

Segment	1	2	3	4	5	6	7	8	9
S1	T1	T2	T3	T4	T5	T6			
S2		T1	T2	T3	T4	T5	T6		
S3			T1	T2	T3	T4	T5	T6	
S4				T1	T2	T3	T4	T5	T6

The first result appears after k cycles, then one result finishes every cycle.

Speedup

With k segments, n tasks and clock period t_p : pipeline time = $(k + n - 1) \cdot t_p$. Non-pipelined time = $n \cdot t_n$, where $t_n = k \cdot t_p$ if the delays are equal.

$S = n \cdot t_n / ((k + n - 1) \cdot t_p)$, which tends to k as n grows. Here $k = 4$, $n = 6$ gives $S = 24/9 \approx 2.67$, and $n = 100$ gives $S = 400/103 \approx 3.88$.

Arithmetic pipeline (floating-point add)



Four segments, with a register between each pair.

Instruction pipeline

Typical four-segment version: **FI** fetch instruction, **DA** decode and calculate the effective address, **FO** fetch operand, **EX** execute. Ideally a new instruction enters every cycle.

Pipeline hazards

Hazard	Cause	Remedies
Resource (structural) conflict	Two segments need the same resource, e.g. memory for FI and FO	Separate instruction and data memories, or duplicate units

Data dependency	An instruction needs a result that a previous one has not yet produced	Hardware interlock (stall), operand forwarding, compiler reordering, delayed load
Branch difficulty	A branch changes the PC, so prefetched instructions are wrong	Prefetch both paths, branch target buffer, branch prediction, delayed branch, loop buffer

Exam tip: for pipeline questions always use $\text{cycles} = k + n - 1$ and $\text{speedup} = n \cdot k / (k + n - 1)$ when segment delays are equal.